

ShopEZ is our hypothetical online retail platform that strives to promote efficiency in managing customer orders, inventory, payments, and shipping. To ensure that the platform is scalable and resilient, a Microservices Architecture combined with an Event-Driven Architecture is used.



• Microservices Architecture:

- Service Separation
 - User Service: Handles its own authentication, profile settings, and customer preference settings.
 - Catalog & Inventory Service: Gives capabilities related to the management of product listings, stock levels, and updates.
 - Order Service: Responsible for customer order processing and order validation and status management.
 - Payment Service: Has capabilities for processing payments and communicating with external payment gateways.
 - Shipping Service: Responsible for logistics and tracking of shipments.
- Benefits: Each service can be deployed and scaled independently. This also affords ShopEZ the luxury of changing or scaling only one part of a system with no impact on other services.

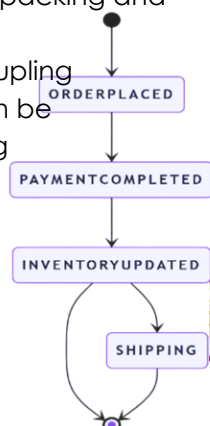
• Improvement of Design using SOLID Principles:



- Single Responsibility Principle: Each microservice (Order, Payment) concerns itself with a single business capability; hence, this design favors maintenance and testing.
- Open/Closed Principle: Your services should be extendable. You can add a new feature with minimal change, i.e., adding or extending the services for handling discounts and user reviews.
- Liskov Substitution Principle: Any replacement or upgrading of particular services should respect its contract so that the behavior of the whole system is unaffected.
- Interface Segregation Principle: Every service guarantees its own API contract without any unnecessary information. Thus, client services access only those endpoints they actually require.
- Dependency Inversion Principle: The services depend upon abstraction (events and contracts) rather than concrete implementations, which add to their flexibility and testability.

• Event-Driven Architecture:

- Event Flow:
 - When a customer generates an order, an OrderPlaced event is emitted from the Order Service.
 - The Payment Service will listen for this event to process the payment. After successful payment, it will publish a PaymentCompleted event.
 - The Inventory Service will receive a PaymentCompleted event to decrement the stock for the purchased items and will publish an InventoryUpdated event.
 - Finally, the Shipping Service will listen to the InventoryUpdated event for the packing and shipping process.
- Benefits: The EDA promotes loose coupling between services such that data can be transferred in real time, thus improving responsiveness.



• Observing DRY and KISS Principles:

- DRY (Don't Repeat Yourself):
 - Centralize common functionality (logging, authentication, configuration) using shared libraries or common services to prevent duplicate code in each microservice.
 - Utilize consistent naming for events and similar structures for all payloads to avoid repeat coding in event handling.
- KISS (Keep It Simple, Stupid):
 - Design each service for a particular, simplified task to reduce complexities.
 - Use straightforward, well-documented API endpoints, and minimize unnecessary abstractions to keep the overall design accessible.

